

Trees and rules

6

CHAPTER OUTLINE

6.1 Decision Trees	210
Numeric Attributes	210
Missing Values	212
Pruning	213
Estimating Error Rates	215
Complexity of Decision Tree Induction	217
From Trees to Rules	219
C4.5: Choices and Options	219
Cost-Complexity Pruning	220
Discussion	221
6.2 Classification Rules	221
Criteria for Choosing Tests	222
Missing Values, Numeric Attributes	223
Generating Good Rules	224
Using Global Optimization	226
Obtaining Rules From Partial Decision Trees	227
Rules With Exceptions	231
Discussion	233
6.3 Association Rules	234
Building a Frequent Pattern Tree	235
Finding Large Item Sets	240
Discussion	241
6.4 WEKA Implementations	242

Decision tree learners are the workhorses of many practical applications of machine learning because they are fast and produce intelligible output that is often surprisingly accurate. This chapter explains how to make decision tree learning robust and versatile enough to cope with the demands of real-world datasets. It shows how to deal with numeric attributes and missing values, and how to

prune away those parts of a tree that do not actually enjoy sufficient support from the data. Our discussion is based on how these issues are addressed in the classic C4.5 algorithm for decision tree learning, but we will also see how cross-validation is used by the famous CART decision tree learner to implement a more robust pruning strategy.

The second topic in this chapter is rule learning. When implemented appropriately, this shares the advantages of decision tree learning, at a somewhat higher cost in runtime, and often yields even more concise classification models. Surprisingly, rules are less popular than decision trees in practice—perhaps because rule learning algorithms are quite heuristic and are not often considered outside the artificial intelligence community. We will see how to make the basic rule learning strategy from [Chapter 4](#), Algorithms: the basic methods, less prone to overfitting, how to deal with missing values and numeric attributes, and how to select and prune good rules. Generating concise and accurate rule sets is trickier than learning concise and accurate decision trees. We will discuss two strategies for achieving this: one based on global optimization of a rule set and another based on extracting rules from partially grown and pruned decision trees. We will also briefly look at the advantages of representing learned knowledge using rules sets with exceptions.

6.1 DECISION TREES

The first machine learning scheme that we will develop in detail, the C4.5 algorithm, derives from the simple divide-and-conquer algorithm for producing decision trees that was described in [Section 4.3](#). It needs to be extended in several ways before it is ready for use on real-world problems. First we consider how to deal with numeric attributes, and, after that, missing values. Then we look at the all-important problem of pruning decision trees, because trees constructed by the basic divide-and-conquer algorithm as described perform well on the training set, but are usually overfitted to it and do not generalize well to independent test sets. We then briefly consider how to convert decision trees to classification rules, and examine the options provided by the C4.5 algorithm itself. Finally, we look at an alternative pruning strategy that is implemented in the famous CART system for learning classification and regression trees.

NUMERIC ATTRIBUTES

The method we described in [Section 4.3](#) only works when all the attributes are nominal, whereas, as we have seen, most real datasets contain some numeric attributes. It is not too difficult to extend the algorithm to deal with these. For a numeric attribute we will restrict the possibilities to a two-way, or binary, split.

Suppose we use the version of the weather data that has some numeric features (Table 1.3). Then, when temperature is being considered for the first split, the temperature values involved are

64	65	68	69	70	71	72	75	80	81	83	85
Yes	No	Yes	Yes	Yes	No	No Yes	Yes Yes	No	Yes	Yes	No

(Repeated values have been collapsed together), and there are only 11 possible positions for the breakpoint—8 if the breakpoint is not allowed to separate items of the same class. The information gain for each can be calculated in the usual way. For example, the test *temperature* < 71.5 produces four *yes*'s and two *no*'s, whereas *temperature* > 71.5 produces five *yes*'s and three *no*'s, and so the information value of this test is

$$\text{Info}([4, 2], [5, 3]) = (6/14) \times \text{info}([4, 2]) + (8/14) \times \text{info}([5, 3]) = 0.939 \text{ bits.}$$

It is common to place numeric thresholds halfway between the values that delimit the boundaries of a concept, although something might be gained by adopting a more sophisticated policy. For example, we will see below that although the simplest form of instance-based learning puts the dividing line between concepts in the middle of the space between them, other methods that involve more than just the two nearest examples have been suggested.

When creating decision trees using the divide-and-conquer method, once the first attribute to split on has been selected, a top-level tree node is created that splits on that attribute, and the algorithm proceeds recursively on each of the child nodes. For each numeric attribute, it appears that the subset of instances at each child node must be re-sorted according to that attribute's values—and, indeed, this is how programs for inducing decision trees are usually written. However, it is not actually necessary to re-sort because the sort order at a parent node can be used to derive the sort order for each child, leading to a speedier implementation, at the expense of storage. Consider the temperature attribute in the weather data, whose sort order (this time including duplicates) is

64	65	68	69	70	71	72	72	75	75	80	81	83	85
7	6	5	9	4	14	8	12	10	11	2	13	3	1

The italicized number below each temperature value gives the number of the instance that has that value: thus instance number 7 has temperature value 64, instance 6 has temperature 65, and so on. Suppose we decide to split at the top level on the attribute *outlook*. Consider the child node for which *outlook* = *sunny*—in fact the examples with this value of *outlook* are numbers 1, 2, 8, 9, and 11. If the italicized sequence is stored with the example set (and a different sequence must be stored for each numeric attribute)—i.e., instance 7 contains a pointer to instance 6, instance 6 points to instance 5, instance 5

points to instance 9, and so on—then it is a simple manner to read off the examples for which *outlook* = *sunny* in order. All that is necessary is to scan through the instances in the indicated order, checking the *outlook* attribute for each and writing down the ones with the appropriate value:

9	8	11	2	1
---	---	----	---	---

Thus repeated sorting can be avoided by storing with each subset of instances the sort order for that subset according to each numeric attribute. The sort order must be determined for each numeric attribute at the beginning; no further sorting is necessary thereafter.

When a decision tree tests a nominal attribute as described in [Section 4.3](#), a branch is made for each possible value of the attribute. However, we have restricted splits on numeric attributes to be binary. This creates an important difference between numeric attributes and nominal ones: once you have branched on a nominal attribute, you have used all the information that it offers, whereas successive splits on a numeric attribute may continue to yield new information. Whereas a nominal attribute can only be tested once on any path from the root of a tree to the leaf, a numeric one can be tested many times. This can yield trees that are messy and difficult to understand because the tests on any single numeric attribute are not located together but can be scattered along the path. An alternative, which is harder to accomplish but produces a more readable tree, is to allow a multiway test on a numeric attribute, testing against several different constants at a single node of the tree. A simpler but less powerful solution is to prediscritize the attribute as described in [Section 8.2](#).

MISSING VALUES

The next enhancement to the decision tree-building algorithm deals with the problems of missing values. Missing values are endemic in real-world datasets. As explained in [Chapter 2](#), Input: concepts, instances, attributes, one way of handling them is to treat them as just another possible value of the attribute; this is appropriate if the fact that the attribute is missing is significant in some way. In that case no further action need be taken. But if there is no particular significance in the fact that a certain instance has a missing attribute value, a more subtle solution is needed. It is tempting to simply ignore all instances in which some of the values are missing, but this solution is often too draconian to be viable. Instances with missing values often provide a good deal of information. Sometimes the attributes whose values are missing play no part in the decision, in which case these instances are as good as any other.

One question is how to apply a decision tree, once it has been constructed, to an instance in which some of the attributes to be tested have missing values. We outlined a solution in [Section 3.3](#). It involves notionally splitting the instance into pieces, using a numeric weighting scheme, and sending part of it down each

branch in proportion to the number of training instances going down that branch. Eventually, the various parts of the instance will each reach a leaf node, and the decisions at these leaf nodes must be recombined using the weights that have percolated to the leaves.

A second question, which precedes the first one because it applies to training, is how to partition the training set once a splitting attribute has been chosen, to allow recursive application of the decision tree formation procedure on each of the daughter nodes. The same weighting procedure is used. Instances for which the relevant attribute value is missing are notionally split into pieces, one piece for each branch, in the same proportion as the known instances go down the various branches. Pieces of the instance contribute to decisions at lower nodes in the usual way through the information gain or gain ratio calculation, except that they are weighted accordingly. The calculations described in [Section 4.3](#) can also be applied to partial instances. Instead of having integer counts, the weights are used when computing both gain figures. Instances may be further split at lower nodes, of course, if the values of other attributes are unknown as well.

PRUNING

Fully expanded decision trees often contain unnecessary structure, and it is generally advisable to simplify them before they are deployed. Now it is time to learn how to prune decision trees.

By building the complete tree and pruning it afterward we are adopting a strategy of *postpruning* (sometimes called *backward pruning*) rather than *prepruning* (or *forward pruning*). Prepruning would involve trying to decide during the tree-building process when to stop developing subtrees—quite an attractive prospect because that would avoid all the work of developing subtrees only to throw them away afterward. However, postpruning does seem to offer some advantages. For example, situations occur in which two attributes individually seem to have nothing to contribute but are powerful predictors when combined—a sort of combination-lock effect in which the correct combination of the two attribute values is very informative whereas the attributes taken individually are not. Most decision tree builders postprune.

Two rather different operations have been considered for postpruning: *subtree replacement* and *subtree raising*. At each node, a learning scheme might decide whether it should perform subtree replacement, subtree raising, or leave the subtree as it is, unpruned. Subtree replacement is the primary pruning operation, and we look at it first. The idea is to select some subtrees and replace them with single leaves. For example, the whole subtree in [Fig. 1.3B](#), involving two internal nodes and four leaf nodes, has been replaced by the single leaf *bad*. This will certainly cause the accuracy on the training set to decrease if the original tree is produced by the decision tree algorithm described previously, because that continues to build the tree until all leaf nodes are pure (or until all attributes have been tested). However, it may increase the accuracy on an independently chosen test set.

When subtree replacement is implemented, it proceeds from the leaves and works back up toward the root. In the Fig. 1.3 example, the whole subtree in (b) would not be replaced at once. First, consideration would be given to replacing the three daughter nodes in the *health plan contribution* subtree with a single leaf node. Assume that a decision is made to perform this replacement—we will explain how shortly. Then, continuing to work back from the leaves, consideration would be given to replacing the *working hours per week* subtree, which now has just two daughter nodes, by a single leaf node. In the Fig. 1.3 example this replacement was indeed made, which accounts for the entire subtree in (b) being replaced by a single leaf marked *bad*. Finally, consideration would be given to replacing the two daughter nodes in the *wage increase 1st year* subtree with a single leaf node. In this case that decision was not made, so the tree remains as shown in Fig. 1.3A. Again, we will examine how these decisions are actually made shortly.

The second pruning operation, subtree raising, is more complex, and it is not clear that it is necessarily always worthwhile. However, because it is used in the influential decision tree building system C4.5, we describe it here. Subtree raising does not occur in the Fig. 1.3 example, so use the artificial example of Fig. 6.1 for illustration. Here, consideration is given to pruning the tree in Fig. 6.1A, and the result is shown in Fig. 6.1B. The entire subtree from C downward has been “raised” to replace the B subtree. Note that although the daughters of B and C are shown as leaves, they can be entire subtrees. Of course, if we perform this raising operation, it is necessary to reclassify the examples at the nodes marked 4 and 5 into the new subtree headed by C. This is why the daughters of that node are marked with primes: 1', 2', and 3'—to indicate that they are not the same as the original daughters 1, 2, and 3 but differ by the inclusion of the examples originally covered by 4 and 5.

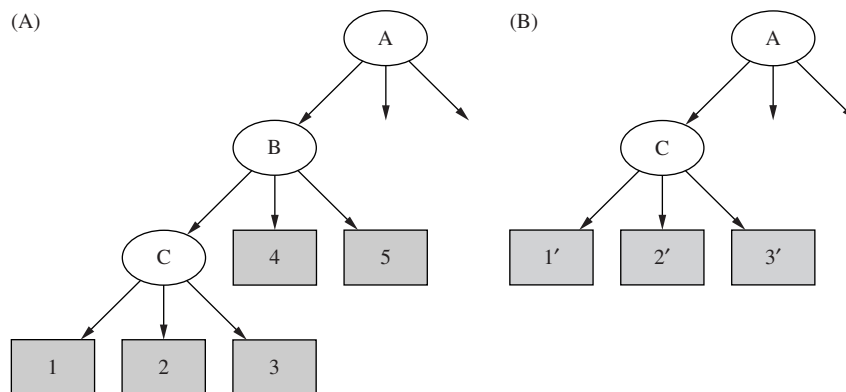


FIGURE 6.1

Example of subtree raising, where node C is “raised” to subsume node B.

Subtree raising is a potentially time-consuming operation. In actual implementations it is generally restricted to raising the subtree of the most popular branch. That is, we consider doing the raising illustrated in Fig. 6.1 provided that the branch from B to C has more training examples than the branches from B to node 4 or from B to node 5. Otherwise, if (e.g.) node 4 were the majority daughter of B, we would consider raising node 4 to replace B and reclassifying all examples under C, as well as the examples from node 5, into the new node.

ESTIMATING ERROR RATES

So much for the two pruning operations. Now we must address the question of how to decide whether to replace an internal node by a leaf (for subtree replacement), or whether to replace an internal node by one of the nodes below it (for subtree raising). To make this decision rationally, it is necessary to estimate the error rate that would be expected at a particular node given an independently chosen test set. We need to estimate the error at internal nodes as well as at leaf nodes. If we had such an estimate, it would be clear whether to replace, or raise, a particular subtree simply by comparing the estimated error of the subtree with that of its proposed replacement. Before estimating the error for a subtree proposed for raising, examples that lie under siblings of the current node—the examples at 4 and 5 of Fig. 6.1—would have to be temporarily reclassified into the raised tree.

It is no use taking the training set error as the error estimate: that would not lead to any pruning because the tree has been constructed expressly for that particular training set. One way of coming up with an error estimate is the standard verification technique: hold back some of the data originally given and use it as an independent test set to estimate the error at each node. This is called *reduced-error* pruning. It suffers from the disadvantage that the actual tree is based on less data.

The alternative is to try to make some estimate of error based on the training data itself. That is what C4.5 does, and we will describe its method here. It is a heuristic based on some statistical reasoning, but the statistical underpinning is rather weak. However, it seems to work well in practice. The idea is to consider the set of instances that reach each node and imagine that the majority class is chosen to represent that node. That gives us a certain number of “errors,” E , out of the total number of instances, N . Now imagine that the true probability of error at the node is q , and that the N instances are generated by a Bernoulli process with parameter q , of which E turn out to be errors.

This is almost the same situation as we considered when looking at the hold-out method in Section 5.2, where we calculated confidence intervals on the true success probability p given a certain observed success rate. There are two differences. One is trivial: here we are looking at the error rate q rather than the success rate p ; these are simply related by $p + q = 1$. The second is more serious: here the figures E and N are measured from the training data, whereas in

Section 5.2 we were considering independent test data instead. Because of this difference we make a pessimistic estimate of the error rate by using the upper confidence limit rather than stating the estimate as a confidence range.

The mathematics involved is just the same as before. Given a particular confidence c (the default figure used by C4.5 is $c = 25\%$), we find confidence limits z such that

$$\Pr \left[\frac{f - q}{\sqrt{q(1 - q)/N}} > z \right] = c,$$

where N is the number of samples, $f = E/N$ is the observed error rate, and q is the true error rate. As before, this leads to an upper confidence limit for q . Now we use that upper confidence limit as a (pessimistic) estimate for the error rate e at the node:

$$e = \frac{f + \frac{z^2}{2N} + z \sqrt{\frac{f}{N} - \frac{f^2}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}.$$

Note the use of the $+$ sign before the square root in the numerator to obtain the upper confidence limit. Here, z is the number of standard deviations corresponding to the confidence c , which for $c = 25\%$ is $z = 0.69$.

To see how all this works in practice, let's look again at the labor negotiations decision tree of Fig. 1.3, salient parts of which are reproduced in Fig. 6.2 with the number of training examples that reach the leaves added. We use the above formula with a 25% confidence figure, i.e., with $z = 0.69$. Consider the lower left

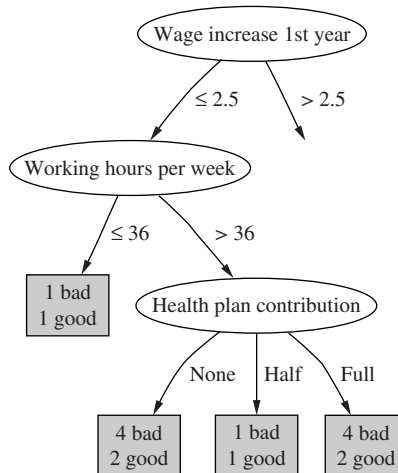


FIGURE 6.2

Pruning the labor negotiations decision tree.

leaf, for which $E = 2$, $N = 6$, and so $f = 0.33$. Plugging these figures into the formula, the upper confidence limit is calculated as $e = 0.47$. That means that instead of using the training set error rate for this leaf, which is 33%, we will use the pessimistic estimate of 47%. This is pessimistic indeed, considering that it would be a bad mistake to let the error rate exceed 50% for a two-class problem. But things are worse for the neighboring leaf, where $E = 1$ and $N = 2$, because the upper confidence limit becomes $e = 0.72$. The third leaf has the same value of e as the first. The next step is to combine the error estimates for these three leaves in the ratio of the number of examples they cover, 6: 2:6, which leads to a combined error estimate of 0.51. Now we consider the error estimate for the parent node, *health plan contribution*. This covers nine bad examples and five good ones, so the training set error rate is $f = 5/14$. For these values, the above formula yields a pessimistic error estimate of $e = 0.46$. Because this is less than the combined error estimate of the three children, they are pruned away.

The next step is to consider the *working hours per week* node, which now has two children that are both leaves. The error estimate for the first, with $E = 1$ and $N = 2$, is $e = 0.72$, while for the second it is $e = 0.46$ as we have just seen. Combining these in the appropriate ratio of 2:14 leads to a value that is higher than the error estimate for the *working hours* node, so the subtree is pruned away and replaced by a leaf node.

The estimated error figures obtained in these examples should be taken with a grain of salt because the estimate is only a heuristic one and is based on a number of shaky assumptions: the use of the upper confidence limit; the assumption of a normal distribution; and the fact that statistics from the training set are used. However, the qualitative behavior of the error formula is correct and the method seems to work reasonably well in practice. If necessary, the underlying confidence level, which we have taken to be 25%, can be tweaked to produce more satisfactory results.

COMPLEXITY OF DECISION TREE INDUCTION

Now that we have learned how to accomplish the pruning operations, we have finally covered all the central aspects of decision tree induction. Let's take stock and examine the computational complexity of inducing decision trees. We will use the standard order notation: $O(n)$ stands for a quantity that grows at most linearly with n , $O(n^2)$ grows at most quadratically with n , and so on.

Suppose the training data contains n instances and m attributes. We need to make some assumption about the size of the tree, and we will assume that its depth is on the order of $\log n$, i.e., $O(\log n)$. This is the standard rate of growth of a tree with n leaves, provided that it remains "bushy" and does not degenerate into a few very long, stringy branches. Note that we are tacitly assuming that most of the instances are different from each other, and—this is almost the same thing—that the m attributes provide enough tests to allow the instances to be differentiated. For example, if there were only a few binary

attributes, they would allow only so many instances to be differentiated and the tree could not grow past a certain point, rendering an “in the limit” analysis meaningless.

The computational cost of building the tree in the first place is

$$O(mn \log n)$$

Consider the amount of work done for one attribute over all nodes of the tree. Not all the examples need to be considered at each node, of course. But at each possible tree depth, the entire set of n instances must be considered. And because there are $\log n$ different depths in the tree, the amount of work for this one attribute is $O(n \log n)$. At each node all attributes are considered, so the total amount of work is $O(mn \log n)$.

This reasoning makes some assumptions. If some attributes are numeric, they must be sorted, but once the initial sort has been done there is no need to re-sort at each tree depth if the appropriate algorithm is used (described earlier). The initial sort takes $O(n \log n)$ operations for each of up to m attributes: thus the above complexity figure is unchanged. If the attributes are nominal, all attributes do *not* have to be considered at each tree node—because attributes that are used further up the tree cannot be reused. However, if attributes are numeric, they can be reused and so they have to be considered at every tree level.

Next, consider pruning by subtree replacement. First an error estimate must be made for every tree node. Provided that counts are maintained appropriately, this is linear in the number of nodes in the tree. Then each node needs to be considered for replacement. The tree has at most n leaves, one for each instance. If it were a binary tree, each attribute being numeric or two-valued, that would give it $2n - 1$ nodes; multiway branches would only serve to decrease the number of internal nodes. Thus the complexity of subtree replacement is

$$O(n)$$

Finally, subtree lifting has a basic complexity equal to subtree replacement. But there is an added cost because instances need to be reclassified during the lifting operation. During the whole process, each instance may have to be reclassified at every node between its leaf and the root, i.e., as many as $O(\log n)$ times. That makes the total number of reclassifications $O(n \log n)$. And reclassification is not a single operation: one that occurs near the root will take $O(\log n)$ operations, and one of average depth will take half of this. Thus the total complexity of subtree lifting is as follows:

$$O(n(\log n)^2).$$

Taking into account all these operations, the full complexity of decision tree induction is

$$O(mn \log n) + O(n(\log n)^2).$$

FROM TREES TO RULES

It is possible to read a set of rules directly off a decision tree, as noted in [Section 3.4](#), by generating a rule for each leaf and making a conjunction of all the tests encountered on the path from the root to that leaf. This produces rules that are unambiguous in that it does not matter in what order they are executed. However, the rules are more complex than necessary.

The estimated error rate described previously provides exactly the mechanism necessary to prune the rules. Given a particular rule, each condition in it is considered for deletion by tentatively removing it, working out which of the training examples are now covered by the rule, calculating from this a pessimistic estimate of the error rate of the new rule, and comparing this with the pessimistic estimate for the original rule. If the new rule is better, delete that condition and carry on, looking for other conditions to delete. Leave the rule when there are no conditions left that will improve it if they are removed. Once all rules have been pruned in this way, it is necessary to see if there are any duplicates and remove them from the rule set.

This is a greedy approach to detecting redundant conditions in a rule, and there is no guarantee that the best set of conditions will be removed. An improvement would be to consider all subsets of conditions, but this is usually prohibitively expensive. Another solution might be to use an optimization technique such as simulated annealing or a genetic algorithm to select the best version of this rule. However, the simple greedy solution seems to produce quite good rule sets.

The problem, even with the greedy method, is computational cost. For every condition that is a candidate for deletion, the effect of the rule must be reevaluated on all the training instances. This means that rule generation from trees tends to be very slow, and [Section 6.2: Classification Rules](#) describes much faster methods that generate classification rules directly without forming a decision tree first.

C4.5: CHOICES AND OPTIONS

C4.5 works essentially as described in the “From Trees to Rules” sections. The default confidence value is set at 25% and works reasonably well in most cases; possibly it should be altered to a lower value, which causes more drastic pruning, if the actual error rate of pruned trees on test sets is found to be much higher than the estimated error rate. There is one other important parameter whose effect is to eliminate tests for which almost all of the training examples have the same outcome. Such tests are often of little use. Consequently tests are not incorporated into the decision tree unless they have at least two outcomes that have at least a minimum number of instances. The default value for this minimum is 2, but it is controllable and should perhaps be increased for tasks that have a lot of noisy data.

Another heuristic in C4.5 is that candidate splits on numeric attributes are only considered if they cut off a certain minimum number of instances: at least 10% of the average number of instances per class at the current node, or 25 instances, whichever value is smaller (but the above minimum, 2 by default, is also enforced).

C4.5 Release 8, the version of C4.5 implemented in the Weka software, includes an MDL-based adjustment to the information gain for splits on numeric attributes. More specifically, if there are S candidate splits on a certain numeric attribute at the node currently considered for splitting, $\log_2(S)/N$ is subtracted from the information gain, where N is the number of instances at the node. This heuristic is designed to prevent overfitting. The information gain may be negative after subtraction, and tree growing will stop if there are no attributes with positive information gain—a form of prepruning. We mention this here because it can be surprising to obtain a pruned tree even if postpruning has been turned off!

Finally, C4.5 does not actually place the split point for a numeric attribute halfway between two values. Once a split has been chosen, the entire training set is searched to find the greatest value for that attribute that does not exceed the provisional split point, and this becomes the actual split point. This adds a quadratic term $O(n^2)$ to the time complexity because it can happen at any node, which we have ignored above.

COST-COMPLEXITY PRUNING

As mentioned above, the postpruning method in C4.5 is based on shaky statistical assumptions and it turns out that it often does not prune enough. On the other hand, it is very fast and thus popular in practice. However, in many applications it is worthwhile expending more computational effort to obtain a more compact decision tree. Experiments have shown that C4.5's pruning method can yield unnecessary additional structure in the final tree: tree size continues to grow when more instances are added to the training data even when this does not further increase performance on independent test data. In that case the more conservative *cost-complexity pruning* method from the CART tree learning system may be more appropriate.

Cost-complexity pruning is based on the idea of first pruning those subtrees that, relative to their size, lead to the smallest increase in error on the training data. The increase in error is measured by a quantity α that is defined to be the average error increase per leaf of the subtree concerned. By monitoring this quantity as pruning progresses, the algorithm generates a sequence of successively smaller pruned trees. In each iteration it prunes all subtrees that exhibit the smallest value of α amongst the remaining subtrees in the current version of the tree.

Each candidate tree in the resulting sequence of pruned trees corresponds to one particular threshold value α_i . The question becomes: which tree should be chosen as the final classification model? To determine the most predictive tree, cost-complexity pruning either uses a holdout set to estimate the error rate of each tree, or, if data is limited, employs cross-validation.

Using a holdout set is straightforward. However, cross-validation poses the problem of relating the α values observed in the sequence of pruned trees for training fold k of the cross-validation to the α values from the sequence of trees for the full dataset: these values are usually different. This problem is solved by first computing the geometric average of α_i and α_{i+1} for tree i from the full dataset. Then, for each fold k of the cross-validation, the tree that exhibits the largest α value smaller than this average is picked. The average of the error estimates for these trees from the k folds, estimated from the corresponding test datasets, is the cross-validation error for tree i from the full dataset.

DISCUSSION

Top-down induction of decision trees is probably the most extensively researched method of machine learning used in data mining. Researchers have investigated a panoply of variations for almost every conceivable aspect of the learning process—e.g., different criteria for attribute selection or modified pruning methods. However, they are rarely rewarded by substantial improvements in accuracy over a spectrum of diverse datasets. As discussed above, the pruning method used by the CART system for learning decision trees (Breiman et al., 1984) can often produce smaller trees than C4.5's pruning method. This has been investigated empirically by Oates and Jensen (1997).

The decision tree program C4.5 and its successor C5.0 were devised by Ross Quinlan over a 20-year period beginning in the late 1970s. A complete description of C4.5, the early 1990s version, appears as an excellent and readable book (Quinlan, 1993), along with the full source code. The MDL heuristic for C4.5 Release 8 is described by Quinlan (1996). The more recent version, C5.0, is also available as open-source code.

In our description of decision trees, we have assumed that only one attribute is used to split the data into subsets at each node of the tree. However, it is possible to allow tests that involve several attributes at a time. For example, with numeric attributes each test can be on a linear combination of attribute values. Then the final tree consists of a hierarchy of linear models of the kind described in [Section 4.6](#), and the splits are no longer restricted to being axis-parallel. The CART system has the option of generating such tests. They are often more accurate and smaller than standard trees, but take much longer to generate and are also more difficult to interpret. We briefly mention one way of generating them under principal component analysis in [Section 8.3](#).

6.2 CLASSIFICATION RULES

We call the basic covering algorithm for generating rules that was described in [Section 4.4](#) a separate-and-conquer technique because it identifies a rule that

covers instances in the class (and excludes ones not in the class), separates them out, and continues on those that are left. Such algorithms have been used as the basis of many systems that generate rules. There we described a simple correctness-based measure for choosing what test to add to the rule at each stage. However, there are many other possibilities, and the particular criterion that is used has a significant effect on the rules produced. We examine different criteria for choosing tests in this section. We also look at how the basic rule-generation algorithm can be extended to more practical situations by accommodating missing values and numeric attributes.

But the real problem with all these rule-generation schemes is that they tend to overfit the training data and do not generalize well to independent test sets, particularly on noisy data. To be able to generate good rule sets for noisy data, it is necessary to have some way of measuring the real worth of individual rules. The standard approach to assessing the worth of rules is to evaluate their error rate on an independent set of instances, held back from the training set, and we explain this next. After that, we describe two industrial-strength rule learners: one that combines the simple separate-and-conquer technique with a global optimization step, and another one that works by repeatedly building partial decision trees and extracting rules from them. Finally, we consider how to generate rules with exceptions, and exceptions to the exceptions.

CRITERIA FOR CHOOSING TESTS

When we introduced the basic rule learner in [Section 4.4](#), we had to figure out a way of deciding which of many possible tests to add to a rule to prevent it from covering any negative examples. For this we used the test that maximizes the ratio

$$p/t$$

where t is the total number of instances that the new rule will cover, and p is the number of these that are positive—i.e., belong to the class in question. This attempts to maximize the “correctness” of the rule on the basis that the higher the proportion of positive examples it covers, the more correct a rule is. One alternative is to calculate an information gain:

$$p \left[\log \frac{p}{t} - \log \frac{P}{T} \right],$$

where p and t are the number of positive instances and the total number of instances covered by the new rule, as before, and P and T are the corresponding number of instances that satisfied the rule *before* the new test was added. The rationale for this is that it represents the total information gained regarding the current positive examples, which is given by the number of them that satisfy the new test, multiplied by the information gained regarding each one.

The basic criterion for choosing a test to add to a rule is to find one that covers as many positive examples as possible, while covering as few negative examples as possible. The original correctness-based heuristic, which is just the percentage of positive examples among all examples covered by the rule, attains a maximum when no negative examples are covered regardless of the number of positive examples covered by the rule. Thus a test that makes the rule exact will be preferred to one that makes it inexact, no matter how few positive examples the former rule covers, nor how many positive examples the latter covers. For example, if we can choose between a test that covers one example, which is positive, this criterion will prefer it over a test that covers 1000 positive examples along with one negative one.

The information-based heuristic, on the other hand, places far more emphasis on covering a large number of positive examples regardless of whether the rule so created is exact. Of course, both algorithms continue adding tests until the final rule produced is exact, which means that the rule will be finished earlier using the correctness measure, whereas more terms will have to be added if the information-based measure is used. Thus the correctness-based measure might find special cases and eliminate them completely, saving the larger picture for later (when the more general rule might be simpler because awkward special cases have already been dealt with), whereas the information-based one will try to generate high-coverage rules first and leave the special cases until later. It is by no means obvious that either strategy is superior to the other at producing an exact rule set. Moreover, the whole situation is complicated by the fact that, as described below, rules may be pruned and inexact ones tolerated.

MISSING VALUES, NUMERIC ATTRIBUTES

As with divide-and-conquer decision tree algorithms, the nasty practical considerations of missing values and numeric attributes need to be addressed. In fact, there is not much more to say. Now that we know how these problems can be solved for decision tree induction, appropriate solutions for rule induction are easily given.

When producing rules using covering algorithms, missing values can best be treated as though they don't match any of the tests. This is particularly suitable when a decision list is being produced because it encourages the learning algorithm to separate out positive instances using tests that are known to succeed. It has the effect that either instances with missing values are dealt with by rules involving other attributes that are not missing, or any decisions about them are deferred until most of the other instances have been taken care of, at which time tests will probably emerge that involve other attributes. Covering algorithms for decision lists have a decided advantage over decision tree algorithms in this respect: tricky examples can be left until late in the process, at which time they will appear less tricky because most of the other examples have already been classified and removed from the instance set.

Numeric attributes can be dealt with in exactly the same way as they are for trees. For each numeric attribute, instances are sorted according to the attribute's value and, for each possible threshold, a binary less-than/greater-than test is considered and evaluated in exactly the same way that a binary attribute would be.

GENERATING GOOD RULES

Suppose you don't want to generate perfect rules that guarantee to give the correct classification on all instances in the training set, but would rather generate "sensible" ones that avoid overfitting the training set and thereby stand a better chance of performing well on new test instances. How do you decide which rules are worthwhile? How do you tell when it becomes counterproductive to continue adding terms to a rule to exclude a few pesky instances of the wrong type, all the while excluding more and more instances of the right type, too?

Let's look at a few examples of possible rules—some good and some bad—for the contact lens problem in [Table 1.1](#). Consider first the rule

```
If astigmatism = yes and tear production rate = normal
    then recommendation = hard.
```

This gives a correct result for four out of the six cases that it covers; thus its success fraction is 4/6. Suppose we add a further term to make the rule a "perfect" one:

```
If astigmatism = yes and tear production rate = normal
    and age = young then recommendation = hard.
```

This improves accuracy to 2/2. Which rule is better? The second one is more accurate on the training data but covers only two cases, whereas the first one covers six. It may be that the second version is just overfitting the training data. For a practical rule learner we need a principled way of choosing the appropriate version of a rule, preferably one that maximizes accuracy on future test data.

Suppose we split the training data into two parts that we will call a *growing set* and a *pruning set*. The growing set is used to form a rule using the basic covering algorithm. Then a test is deleted from the rule, and the effect is evaluated by trying out the truncated rule on the pruning set and seeing whether it performs better than the original rule. This pruning process repeats until the rule cannot be improved by deleting any further tests. The whole procedure is repeated for each class, obtaining one best rule for each class, and the overall best rule is established by evaluating the rules on the pruning set. This rule is then added to the rule set, the instances it covers removed from the training data—from both growing and pruning sets—and the process is repeated.

Why not do the pruning as we build the rule up, rather than building up the whole thing and then throwing parts away? That is, why not preprune rather than postprune? Just as when pruning decision trees it is often best to grow the tree to